Care Access Navigator

Incident Response 1 — Task 5: IR Documentation & Reporting

Host IR playbook, an incident tracking system, the incident report for IR1-2026-002, and a Parrot OS IR tools guide

Document ID	CAN-IR1-T5-DOC-002
Author	Akhila Chennamaneni
Date	November 3, 2026
Deliverables	`playbook/CAN-IR-PB-02_Host_Incident_Response_Playbook.md` · `tracker/irtrack.py` + `incidents.db` · incident report (§4) · `tools/PARROT_IR_TOOLS.md`
Tested on	IR1-2026-002 (SEV-1), run end to end in Tasks 3, 4 and 6; the playbook and tools guide quote the commands that were actually run

Summary

This task turns the work of Tasks 1–4 and 6 into documents a hospital security team could use. The **playbook** (CAN-IR-PB-02) sets severities, roles and a first-15-minutes checklist, and gives six runbooks for the incidents this environment actually sees. Each runbook lists Parrot OS commands and evidence steps. The **tracking system** is a working tool, not a spreadsheet: a SQLite database with a command-line interface. It enforces a forward-only status workflow, measures response times against the severity targets, and exports the incident register. It holds 4 incidents. The **incident report** for IR1-2026-002 follows the NIST SP 800-61 structure and includes the HIPAA breach risk assessment. The **tools guide** documents every tool used, with the exact command and its pitfalls.

1. How this report meets the Task 5 requirements

Requirement	Where	Artifact
IR playbook: response procedures for common incidents	§2 (RB-01–06)	playbook/CAN-IR-PB-02_*.md
Playbook: tool-specific commands for Parrot OS	§2 runbooks, §2 section 5	same
Playbook: evidence collection steps	§2 section 4, RB-02 C2–C5	same
Incident tracking system implemented	§3	tracker/irtrack.py, incidents.db, load_cases.py
Tracking: incident details, response actions, timeline of events	§3.2, §4	tables incident / action / event / evidence
Complete incident report for 1 event, standard format	§4	IR1-2026-002 (NIST SP 800-61r2 layout)
Documentation of IR tools and procedures in Parrot OS	§5	tools/PARROT_IR_TOOLS.md

2. Incident response playbook (CAN-IR-PB-02)

Rendered from `playbook/CAN-IR-PB-02_Host_Incident_Response_Playbook.md`. The Markdown file is the controlled copy.

Field	Value
Document ID	CAN-IR-PB-02, version 2.0
Owner	Akhila Chennamaneni, IR analyst
Date	2026-11-02 (tested end to end on IR1-2026-002)
Applies to	Linux hosts at HealthLine Regional Medical Center (HRMC) (lab): the parrot-vm EHR host, the Parrot OS analyst workstation, and any Debian-family server, virtualised under VirtualBox (lab: QEMU)
Companion	CAN-IR-PB-01, the Cloud IR Playbook (`cloud_security_1/can-secops/IR-Playbook.md`), which covers Azure
Basis	NIST SP 800-61r2 (lifecycle), NIST SP 800-86 and RFC 3227 (evidence), HIPAA Security Rule §164.308(a)(6) and the Breach Notification Rule §164.400-414

0. Severity, roles and records

Severity is **Impact x Urgency** (each 1-3), as used in IR1 Task 2:

Score	Severity	Start response	Escalate to	Examples
9	**SEV-1 Critical**	15 min	Incident Commander + Privacy Officer (HIPAA breach risk assessment; 60-day notice clock)	ePHI leaving a host; ePHI read through a role the user should not have
6	**SEV-2 High**	1 hour	Security lead, IAM owner	Unapproved root/sudo grant; confirmed account takeover without data access
3-4	**SEV-3 Medium**	1 business day	SecOps on-call	Blocked attempts by a leaver; blocked brute force
1-2	**SEV-4 Low**	Next review	(none)	Single failed login; informational alert

Role	Responsibility
Incident Commander (IC)	Declares severity, authorises containment that affects service, owns communications
IR analyst (responder)	Runs this playbook, records every command, owns evidence and chain of custody
System owner (it_admin)	Is told before isolation or service shutdown; leads recovery
IAM owner	Applies identity changes (JML corrections, disables, key removal) in Entra ID, Keycloak and on hosts
Privacy Officer	Runs the HIPAA four-factor breach risk assessment and notifications

Every incident gets an ID `IR1-YYYY-NNN` and a record in the tracker (`tracker/irtrack.py new ...`). Record status changes, actions and evidence there as you go, not afterwards.

1. Lifecycle at a glance

Phase	Goal	Key actions	Main tools
Detect	Know something is wrong	Wazuh alert, user report, audit finding	Wazuh, ehr-audit.log, auditd
Triage	Real? How bad?	Validate the alert on the host; set severity; open the tracker record	ausearch, ss, ps, tcpdump
Contain	Stop the harm, keep the evidence	Snapshot, preserve volatile data, block, isolate, shut down	VBoxManage, iptables, ip, systemctl
Preserve	Evidence that holds up	Order of volatility, hash at both ends, chain of custody	tcpdump, gcore, dc3dd, sha256sum
Analyse	What happened, how, who	Memory, disk, logs, timeline	Volatility 3, Sleuth Kit, mactime, timeline script
Eradicate	Remove the cause and the artifacts	Restore clean state; fix the root cause	VBoxManage snapshot, usermod, gpasswd
Recover	Service back, proven safe	Network and services back; validation checklist, twice	validate_recovery.sh
Learn	Do better next time	RCA, report, playbook update	tracker report, this playbook

2. First 15 minutes (any incident)

- **1.** Open the tracker record: `irtrack.py new IR1-2026-NNN --title ... --severity SEV-n --detected <UTC> --source <alert> ...`
- **2.** Note the time on the suspect host against a trusted clock (`date -u` on both) before reading any timestamp.
- **3.** Validate the alert on the host: read the raw log line that fired it. Do not trust the alert text alone.
- **4.** Set severity with the matrix in section 0. For SEV-1 or SEV-2, page the IC now.
- **5.** Decide whether data is leaving or harm is ongoing. If yes, go straight to RB-02 steps C1-C3.
- **6.** Do **not** reboot, shut down, or run cleanup on the suspect host: memory and live connections are evidence.

3. Runbooks for common incidents

RB-01 Unauthorized ePHI access (stale or excessive privilege)

Item	Detail
Trigger	Wazuh 100221 (mover views ePHI), ehr-audit `decision=ALLOW` by a user whose JML role no longer allows it, auditd key `ehr-read`
Default severity	SEV-1 if a record was read; SEV-2 if only attempted
Contain	Remove the stale group (`gpasswd -d <user> clinical`); lock the account if intent is unclear (`usermod -L -e 1 <user>`); revoke IdP sessions in Keycloak/Entra

Item	Detail
Collect	`ehr-audit.log`, `ausearch -k ehr-read -i`, `id <user>`, `/etc/group`, the JML ticket (JML-LOG.csv), all with SHA-256
Check	Was the record copied or sent anywhere? Look for new files (`find /home -newermt`), outbound connections (`ss -tnp`), a running process. If yes, go to RB-02
Eradicate / recover	Fix the JML step that left the group behind; re-run the access review for all movers
Notify	Privacy Officer: four-factor breach risk assessment

RB-02 Data exfiltration in progress (tested on IR1-2026-002)

Run steps C1-C8 in this order. Each step is reversible until C7.

Step	Action	Commands (Parrot OS / VirtualBox)
C1	Live snapshot, the undo point	`VBoxManage snapshot <vm> take ir-pre-containment --live`
C2	Preserve volatile data: packets, processes, connections, persistence	`timeout 25 tcpdump -i <if> -w /dev/shm/live.pcap host <IOC>`; `ps -eo pid,ppid,user,lstart,cmd --forest`; `ss -tunap`; `lsof -nP -i`; `cat /proc/<pid>/{cmdline,environ}`; `crontab -l -u <user>`
C3	**Block the traffic** and keep the process alive	`iptables -N IR-EGRESS-BLOCK`; `iptables -A IR-EGRESS-BLOCK -j LOG --log-prefix "IR-BLOCK " --log-uid`; `iptables -A IR-EGRESS-BLOCK -j DROP`; `iptables -I OUTPUT 1 -d <IOC> -j IR-EGRESS-BLOCK`
C4	Preserve memory: process core and full RAM	`gcore -o /dev/shm/proc <pid>`; `VBoxManage debugvm <vm> dumpvmcore --filename=<case>.elf`
C5	Preserve file-system artifacts	`stat` first, then `cp -a --parents <files> /dev/shm/art`, `sha256sum`, `tar -C /dev/shm/art -cf - . | ssh forensic 'cat > art.tar'`
C6	**Isolate the host**: cable out, quarantine adapter in, default-deny firewall	`VBoxManage controlvm <vm> setlinkstate1 off`; `VBoxManage controlvm <vm> setlinkstate2 on` (adapter 2 on internal network quarantine); `ip link set <if0> down`; `ip addr add <q-ip>/24 dev <if1>`; `iptables -P INPUT DROP; iptables -P OUTPUT DROP; iptables -A INPUT -i <if1> -s <forensic-ws> -p tcp --dport 22 -j ACCEPT`
C7	**Shut down** the process, persistence, exposed services; lock the account	`pkill -KILL -f <proc>`; `systemctl stop cron && systemctl mask cron`; `systemctl stop <svc> && systemctl disable <svc>`; `usermod -L -e 1 <user>`; `gpasswd -d <user> clinical`; move `authorized_keys` to `/root/ir-quarantine/`

Step	Action	Commands (Parrot OS / VirtualBox)
C8	Verify with the same probes as before; snapshot	re-run the reachability probes; `VBoxManage snapshot <vm> take ir-contained --live`

If the transfer is large or ongoing, the IC may order **C3 before C2**: block first, capture after.

RB-03 Brute force and credential compromise

Item	Detail
Trigger	Wazuh 5710/5712/5760 (sshd failures), 100101 (brute force), 100102 (success after failures), 100110 (privilege escalation after login)
Default severity	SEV-3 if all attempts failed; **SEV-1/2 if a login succeeded** after failures
Contain	Block the source IP (`iptables -I INPUT -s <ip> -j DROP`); if a login succeeded: lock the account, kill its sessions (`pkill -KILL -u <user>`), rotate its credentials and keys
Collect	`auth.log`, `last -F`, `lastb -F`, `journalctl -u ssh`, sshd `LogLevel VERBOSE` key fingerprints, Wazuh alerts.json
Check	What did the session do? `ausearch -ua <uid> -i`, bash history, new files, cron, keys
Eradicate / recover	Enforce key-only SSH, MFA at the IdP, lockout (Keycloak `bruteForceProtected=true`, finding F-02)

RB-04 Leaver or disabled account activity

Item	Detail
Trigger	Wazuh 100210/100211 (CDB list `jml-leavers`), IdP sign-in by a disabled user
Default severity	SEV-3 if blocked; SEV-2 if any access succeeded
Contain	Confirm the account is disabled everywhere: Entra ID, Keycloak and **local hosts** (`passwd -S`, `chage -l`, DenyGroups)
Collect	Correlate the client log with the server log by time and SSH key fingerprint (IR1 Task 2 §6, Task 4 §7.2)
Eradicate	Remove standing credentials on every host: `authorized_keys`, local passwords, sudoers entries
Lesson (IR1-2026-001)	Disabling at the IdP does not remove host SSH keys. The leaver step must include host keys

RB-05 Unauthorized privilege escalation

Item	Detail
Trigger	Wazuh 100230 (added to sudo/adm), 100231 (root by a role with no root entitlement), auditd key `sudoers`
Default severity	SEV-2
Contain	`gpasswd -d <user> sudo`; remove any `/etc/sudoers.d/` file not backed by a change record

Item	Detail
Collect	`auth.log` (usermod, sudo COMMAND=), `getent group sudo`, `/etc/sudoers.d/*`, the change ticket (or its absence)
Rule (IR1-2026-003)	Every sudo grant needs a change record and an expiry. Grants without one are removed first and justified after

RB-06 Malware or persistence on a host

Item	Detail
Trigger	auditd `cron-persist`/`user-exec`, unknown process with network activity, new systemd unit, new SSH key
Default severity	SEV-2; SEV-1 if ePHI is reachable from the account
Contain	RB-02 C1-C4 first (memory before kill); then C6
Collect	Process core, RAM image, `crontab -l` for every user, `systemctl list-unit-files`, `find / -newermt`, SUID list, `/proc/<pid>/exe` hash
Analyse	Volatility 3: `linux.pslist`, `linux.psaux`, `linux.malfind`, `linux.check_modules`, `linux.check_syscall`, `linux.sockstat`
Eradicate	Restore from a known-good snapshot or golden image; do not "clean" a compromised host in place

4. Evidence collection procedure (all incidents)

- **1. Clock first.** Record host UTC against a trusted clock.
- **2. Order of volatility.** Memory and network state, then processes, then logs, then disk.
- **3. Do not write to the suspect disk.** Use `/dev/shm` or stream over SSH to the forensic workstation.
- **4. Hash at both ends.** `sha256sum` on the source, then on the copy; record both.
- **5. Freeze live logs** before hashing them (`cp -a` to RAM), or the hash will not match the copy.
- **6. Write-block disks.** Seize the virtual disk from a snapshot at the hypervisor (`VBoxManage clonemedium disk <vdi> case.raw --format RAW`), attach it read-only, image it with `dc3dd hash=sha256 log=...`.
- **7. Chain of custody.** Give each item an ID (E01, E02, ...) and log every acquire, transfer, store, verify and examine event with a hash check (`chain_of_custody.py`).
- **8. Store and restrict.** Keep the case folder on an encrypted volume and set images read-only. Never put real ePHI in a code repository.

5. Host isolation in VirtualBox

Need	VirtualBox	Inside Parrot OS
Undo point before changes	`VBoxManage snapshot <vm> take <name> --live`	(none)
Pull the network cable	`VBoxManage controlvm <vm> setlinkstate1 off`	`ip link set <if> down`
Move to a quarantine segment	`VBoxManage controlvm <vm> nic2 intnet quarantine` then `setlinkstate2 on`	`ip addr add`, `ip route replace default via`

Need	VirtualBox	Inside Parrot OS
Capture traffic outside the guest	`VBoxManage modifyvm <vm> --nictrace1 on --nictracefile1 nic1.pcap` (VM off)	`tcpdump -i <if> -w file.pcap`
Memory without guest tools	`VBoxManage debugvm <vm> dumpvmcore --filename=<case>.elf`	`gcore <pid>` for one process
Run commands with the network cut	`VBoxManage guestcontrol <vm> run --exe /bin/sh -- -c '<cmd>'`	(none)

6. Recovery and validation

Restore, do not clean. Revert to the last known-good snapshot (`VBoxManage snapshot <vm> restore baseline-clean`) or rebuild from the golden image (verify its hash first). Then **fix the root cause on the restored system**, because the baseline usually still contains it. Validate with the checklist (`validate_recovery.sh`, 31 checks), reboot, and validate again. Take a new snapshot `recovered-validated`. Full procedure: IR1 Task 6.

7. Communication and notification

When	Who	What
SEV-1 declared	IC, Privacy Officer, system owner	Short facts: what, when, systems, data at risk, actions under way
Every 30 min during SEV-1	IC to leadership	Status, next step, decisions needed
ePHI involved	Privacy Officer	Four-factor risk assessment (§164.402): nature/extent of PHI, who received it, whether it was actually acquired or viewed, mitigation. Notify individuals within 60 days of discovery unless low probability of compromise is shown. Under 500 individuals: HHS log within 60 days of year end
Closed	IC	Incident report (tracker record plus report), lessons learned, playbook changes

8. Tracking and reporting

```
irtrack.py new IR1-2026-007 --title "..." --type "..." --severity SEV-2 --detected 2026-11-03T10:00:00Z
    --source "Wazuh 100231" --handler "..." --summary "..."
irtrack.py status IR1-2026-007 Triage --at 2026-11-03T10:04:00Z --note "validated on host"
irtrack.py action IR1-2026-007 --at ... --phase Containment --by "..." --what "gpasswd -d user sudo"
    --result "removed"
irtrack.py event  IR1-2026-007 --at ... --host parrot-vm --source auth.log --what "usermod -aG sudo by
    it_admin"
irtrack.py evidence IR1-2026-007 --item E01 --desc "auth.log" --sha256 ... --location ...
irtrack.py list ; irtrack.py report IR1-2026-007 ; irtrack.py export incidents_register.csv
```

9. Maintenance

Review this playbook after every SEV-1 or SEV-2 and at least every 6 months. Change log:

Version	Date	Change
1.0	2026-10-08	Cloud playbook CAN-IR-PB-01 (Azure)

Version	Date	Change
2.0	2026-11-02	Host playbook: RB-01..06, evidence procedure, VirtualBox isolation, recovery; tested on IR1-2026-002; lessons from IR1-2026-001/005/006 added

3. Incident tracking system (irtrack)

`tracker/irtrack.py` uses only Python's standard library and keeps one SQLite file. That makes it portable (Mac, Windows, Parrot) and simple to audit. It does three things a spreadsheet does not: status can only move forward, every change is time-stamped, and response times are computed and checked against the playbook's severity targets.

3.1 Design

Table	Holds	Key fields
incident	One row per incident	id, title, type, severity, status, detected_utc, source, handler, assets, summary
status_history	Every status change	incident, status, at_utc, note (New → Triage → Contained → Eradicated → Recovered → Closed)
action	Response actions	incident, at_utc, phase, by_whom, what, result
event	Timeline of events	incident, at_utc, host, source, what
evidence	Linked evidence	incident, item (E01…), description, sha256, location

Severity	Start response within	Measured as
SEV-1	15 min	detected_utc → first action
SEV-2	60 min	detected_utc → first action
SEV-3	480 min	detected_utc → first action
SEV-4	5 days	detected_utc → first action

3.2 Using it

tracker/cli_walkthrough.txt - workflow on a scratch database, then the real register

```
# irtrack CLI walk-through on a scratch database (IRTRACK_DB=demo.db) - the real register is incidents.db

$ irtrack.py init
initialised demo.db

$ irtrack.py new IR1-2026-099 --title DEMO: unapproved sudo grant --type Unauthorized privilege escalation --severity
    SEV-2 --detected 2026-11-03T20:00:00Z --source Wazuh 100230 --handler A. Chennamaneni --summary walk-through only

$ irtrack.py status IR1-2026-099 Triage --at 2026-11-03T20:05:00Z --note validated on host

$ irtrack.py action IR1-2026-099 --at 2026-11-03T20:06:00Z --phase Containment --by A. Chennamaneni --what gpasswd -d
    test.clinician sudo --result removed

$ irtrack.py status IR1-2026-099 Contained --at 2026-11-03T20:07:00Z

$ irtrack.py status IR1-2026-099 New --at 2026-11-03T20:08:00Z
IR1-2026-099: status can only move forward (Contained -> New refused)

$ irtrack.py list
ID              SEV    STATUS      DETECTED              RESPOND(min) SLA     CONTAIN(min)  TITLE
IR1-2026-099    SEV-2  Contained   2026-11-03T20:00:00Z          6.0 MET              7.0  DEMO: unapproved sudo
    grant

# the real register
$ irtrack.py list
ID              SEV    STATUS      DETECTED              RESPOND(min) SLA     CONTAIN(min)  TITLE
INC-20261008-01 SEV-2  Closed      2026-10-08T20:22:00Z              n/a                   Rogue NSG backdoor rule +
     storage key listing + SSH brute fo
IR1-2026-003    SEV-2  Closed      2026-11-03T19:30:03Z          4.2 MET              4.2  it_admin added joiner
    Sathya Hasi (test.clinician) to sudo w
IR1-2026-001    SEV-3  Closed      2026-11-03T19:30:31Z          3.7 MET              3.7  Leaver Jeevan Smith
    (test.billing) attempted SSH twice; his
IR1-2026-002    SEV-1  Closed      2026-11-03T19:31:10Z          0.0 MET              4.6  Mover Tara Jones used
    stale clinical access to take patient

$ irtrack.py export incidents_register.csv
exported incidents_register.csv
```

3.3 The incident register

`load_cases.py` rebuilds the database from the evidence of Tasks 1–6, so the register always matches what actually happened. `irtrack.py export` writes `incidents_register.csv`, which keeps the project's existing Tableau columns and adds the response metrics.

ID	Sev	Status	Detected (UTC)	Respond	SLA	Contain	Recover	Act/Evt/Ev
INC-20261008-01	SEV-2	Closed	2026-10-08 20:22	–	n/a	–	–	0/0/0
IR1-2026-003	SEV-2	Closed	2026-11-03 19:30	4.2 m	**MET**	4.2 m	18.8 m	3/0/0
IR1-2026-001	SEV-3	Closed	2026-11-03 19:30	3.7 m	**MET**	3.7 m	18.3 m	3/0/0
IR1-2026-002	SEV-1	Closed	2026-11-03 19:31	0.0 m	**MET**	4.6 m	17.6 m	25/18/15

Register: All three IR1 incidents come from one continuous exercise on one lab (Tasks 1–6), so their detection, containment and recovery times can be compared directly. This register replaces the project's earlier rows, which listed IR1 incidents as ‘Resolved’ before any response had been run. The Tableau extract can be refreshed from this export.

4. Incident report — IR1-2026-002

Format: NIST SP 800-61r2 incident documentation (§3.2.5): summary, details, timeline, scope and impact, detection and analysis, response, evidence, root cause, notifications, recommendations, sign-off.

4.1 Executive summary

On 3 November 2026 a Billing employee whose old clinical access was never removed opened patient record PT-1003, copied it, and started a hidden program that sent it to an outside address every five seconds. The SIEM (Wazuh) raised its patient-record alert 1.3 seconds after the record was opened, and the analyst confirmed the transfer on the host. Within **49 seconds** of detection the transfer was blocked, and within 5 minutes the host was isolated and the account locked. 10 copies of the one record reached the outside address. The host was restored from a clean snapshot, the access gap was fixed, and 34 recovery checks passed before and after a reboot. The record is synthetic; with real data this would be a reportable HIPAA breach of one individual's ePHI.

4.2 Incident details

Field	Value
Incident ID / title	IR1-2026-002 — Mover Tara Jones used stale clinical access to take patient record PT-1003 off the host
Type / category	Unauthorized ePHI access / data exfiltration (insider) (NIST: unauthorized access + data exfiltration; MITRE ATT&CK T1078 Valid Accounts, T1048 Exfiltration over alternative protocol, T1053.003 Cron)
Severity	**SEV-1 Critical** (Impact 3 × Urgency 3 = 9)
Status	Closed 2026-11-03
Detected	2026-11-03T19:31:10Z — Wazuh rule 100221 (agent on parrot-vm) + host triage
Contained / eradicated / recovered	2026-11-03T19:35:47Z / 2026-11-03T19:41:09Z / 2026-11-03T19:48:48Z
Handler / reported by	A. Chennamaneni (IR analyst) / Wazuh rule 100221 on agent parrot-vm (automated)
Systems	parrot-vm (EHR host, Debian 12 / Parrot OS stand-in), 10.0.2.15; EHR portal service; drop site 10.78.99.66 (external, lab)
Accounts	test.nurse — Tara Jones, mover Nurse → Billing (CHG-2026-0201)
Data	1 record (PT-1003, synthetic): SYNTHETIC RECORD PT-1003 | Patient: Test Patient Three | DOB: 1979-11-23 (synthetic) | MRN: SYN-0003 | Dx: type 2 diabetes (synthetic)

4.3 Timeline (UTC, 2026-11-02)

UTC	Event	Source
19:31:06	Tara's first SSH session from the Mac authenticated	ssh_client.log
19:31:06	Parrot sshd accepts test.nurse (same key fingerprint)	auth.log

UTC	Event	Source
19:31:08	Staged copy written (file system)	mactime
19:31:08	PT-1003 viewed through stale clinical group	ehr-audit.log
19:31:08	auditd: patient file read	audit.log
19:31:09	Uploader script written (file system)	mactime
19:31:09	Uploader started (auditd)	audit.log
19:31:10	Wazuh alert 100221 (detection, SEV-1)	alerts.json
19:31:10	First upload received by drop site	uploads.jsonl
19:31:11	Persistence: crontab written (auditd)	audit.log
19:31:12	Triage confirms disclosure in progress	task3 markers.csv
19:31:56	Last upload received by drop site	uploads.jsonl
19:31:59	Egress blocked (C3)	task3 markers.csv
19:32:00	First firewall drop logged	kern.log
19:32:32	Memory preserved (C4)	task3 markers.csv
19:32:52	Host isolated (C6)	task3 markers.csv
19:34:16	Account locked + clinical group removed	auth.log
19:35:47	Contained	task3 markers.csv

4.4 Scope and impact

- **Data:** one patient record, sent 10 times (all byte-identical: SHA-256 `2dfb56c223ff1e45`…) between 19:31:10 and 19:31:56. No other record was read: the EHR audit trail and auditd show one read.
- **Systems:** one host. No lateral movement: the account had no sudo, and auditd recorded 34 commands from it. They touched only its home folder, the one record and its own crontab. Memory analysis found no rootkit (Task 4 §4.1).
- **Service:** the EHR portal on this host was down from containment until recovery (planned trade-off, approved by IC).
- **Business:** potential HIPAA breach notification for one individual; reputational risk; HR matter for the employee.

4.5 Detection and analysis

Wazuh rule 100221 (built in Task 1) fires when an account on the JML movers list reads a clinical record; the agent on parrot-vm delivered the EHR audit line and the alert fired within seconds. On the host, the analyst confirmed the read in the EHR audit trail and auditd, and saw SYN packets to 10.78.99.66:8080 every 5 s from a process owned by test.nurse. That confirmed disclosure in progress, so the incident stayed SEV-1. Later analysis (Task 4) confirmed it from memory, the disk image and a timeline correlated across hosts by SSH key fingerprint.

4.6 Response actions

UTC	Phase	Action
11-03 19:31	Triage	Host triage: record read, SYNs to 10.78.99.66 every 5 s, process owned by test.nurse
11-03 19:31	Containment	C1 live snapshot ir-pre-containment
11-03 19:31	Evidence	C2 tcpdump + volatile state to the case folder
11-03 19:31	Containment	C3 iptables chain IR-EGRESS-BLOCK drops all traffic to 10.78.99.66
11-03 19:32	Evidence	C4 gcore of the uploader + full RAM image
11-03 19:32	Evidence	C5 file artifacts stat'd, frozen, hashed, copied
11-03 19:32	Containment	C6 clinical cable out, quarantine adapter in, default-DROP, Wazuh kept via quarantine forward
11-03 19:34	Containment	C7 kill uploader, mask cron, stop portal, lock account, remove clinical group, restrict sshd
11-03 19:34	Containment	C8 probe matrix re-run
11-03 19:35	Containment	Snapshot ir-contained
11-03 19:37	Analysis	Log analysis + severity classification (Task 2)
11-03 19:37	Evidence / analysis	Intake: Task 3 evidence re-hashed
11-03 19:37	Evidence / analysis	Live response: 24 commands, RAM-only, hashed
11-03 19:37	Evidence / analysis	Volatility 3 on the RAM image; hash unchanged
11-03 19:38	Evidence / analysis	Disk seized from snapshot ir-contained
11-03 19:39	Evidence / analysis	dc3dd image from a read-only source; hashes equal
11-03 19:39	Evidence / analysis	Sleuth Kit: planted files recovered, hash-identical
11-03 19:39	Evidence / analysis	Chain of custody + super-timeline built
11-03 19:40	Eradication / recovery	Evidence gate: all items re-verified before recovery
11-03 19:41	Eradication / recovery	Snapshot pre-recovery; golden image hash verified
11-03 19:41	Eradication / recovery	Restore snapshot baseline-clean; adapters to production wiring
11-03 19:41	Eradication / recovery	recover_harden.sh: JML fix, keys, cron.allow, firewall, services
11-03 19:43	Eradication / recovery	Validation run 1: 34/34 checks, 6 probes
11-03 19:45	Eradication / recovery	Reboot; validation run 2: 34/34
11-03 19:48	Eradication / recovery	Snapshot recovered-validated (new baseline)

4.7 Evidence

Item	Description	SHA-256
E01	Full RAM image of parrot-vm (1 GiB ELF core)	`7010d346a57e2818d14b…`
E02	Process memory core of uploader syncd.py	`1d4ccb577d0dcdd5b1cd…`
E03	Live packet capture, host side (25 s, host 10.78.99.66)	`ef0c2bf5c2d31d938449…`
E04	Hypervisor capture of adapter 1 (clinical), whole run	`3967907c437e37817e20…`
E05	Hypervisor capture of adapter 2 (quarantine)	`f1d996df10dc4c9a3450…`
E06	Volatile state set (processes, connections, suspect process, persistence, system state)	`af5b79881da60707f4a8…`
E07	File-system artifacts tar (12 files incl. staged record, uploader, crontab, logs)	`871eb2e5c6b26ecc7291…`
E08	Drop-site upload log	`9b60c46749c9e3eee0f6…`
E09	macOS SSH client log	`3b023d79493b49ba5493…`
E10	Live-response collection (24 command outputs + collection log)	`fd236a6a68c67e31b967…`
E11	Seized virtual disk, raw (state at snapshot ir-contained)	`fa80be62ad1c05688c43…`
E12	Forensic disk image (dc3dd) of E11	`fa80be62ad1c05688c43…`
E13	Wazuh alerts (SIEM record of the incident, incl. rule 100221 that triggered containment)	`8f44f43e075a53662ed8…`
E14	Recovery validation checklist (checklist_run1.tsv)	`a8a5559bd6c265997468…`
E15	Recovery validation checklist (checklist_run2.tsv)	`a8a5559bd6c265997468…`

Chain of custody: Task 4 §6 (12 items, 51 custody events). All items were re-verified before recovery (Task 6 gate: 100/100 OK).

4.8 Root cause

Direct cause: the mover change CHG-2026-0201 added the Billing group on the Linux host but did not remove the clinical group. The identity provider handled its side automatically (Keycloak removed the Nurse role), but local host groups were outside the JML process. **Contributing:** no host firewall or egress filtering; users could create cron jobs; disabled accounts kept SSH keys; detection raised an alert but containment was manual. Full RCA in Task 6.

4.9 HIPAA breach risk assessment (45 CFR 164.402)

Factor	Assessment	Risk
1. Nature and extent of PHI	Name, DOB, MRN and diagnosis for one patient: directly identifying, clinical	**High**
2. Unauthorized person	Unknown recipient at an external address, set up by a workforce member acting outside her role	**High**
3. Actually acquired or viewed?	Yes: the drop site logged 10 complete copies	**High**
4. Extent mitigated	Host contained; no way to retrieve or confirm destruction at the external address	Medium
Conclusion	Low probability of compromise **cannot** be shown → breach. Notify the individual without unreasonable delay and within 60 days of discovery; record in the HHS annual log (< 500). *Lab exercise: the record is synthetic, so no notice was sent.*	**Notify**

4.10 Recovery and validation

Restored from the pre-incident snapshot, root cause fixed, then validated twice (before and after a reboot): 34/34 and 34/34 checklist items, 6/6 network probes. Details: Task 6.

4.11 Recommendations

#	Recommendation	Owner	Due
1	Extend JML to local host groups and SSH keys; reconcile hosts with the IdP every night	IAM owner	2026-11-16
2	Monthly access review of all movers for 90 days after a move	IAM owner / managers	2026-11-30
3	Host firewall with egress allow-list on every EHR host (done on parrot-vm)	System owner	done / 2026-11-16 for others
4	Auto-contain on rule 100221 + outbound connection: run RB-02 C3 automatically, page the IC	Security lead	2026-12-15
5	Restrict cron to administrators (cron.allow) fleet-wide (done on parrot-vm)	System owner	2026-11-16

4.12 Sign-off

Role	Name	Date
Prepared by (IR analyst)	Akhila Chennamaneni	2026-11-02
Incident Commander	(lab: same person)	2026-11-02
Privacy Officer review	(pending, lab)	

5. IR tools and procedures in Parrot OS

Rendered from `tools/PARROT_IR_TOOLS.md`.

Document CAN-IR-TOOLS-01, version 1.0, 2026-11-02. Every command below was run during IR1 Tasks 1-6 on case IR1-2026-002 (transcripts are cited). All tools are in Parrot OS Security, or in Debian 12 for the lab stand-in, unless marked otherwise. **Run as root only where stated.** Where a command changes the system, the caution column says so.

1. Live response: system state, processes, network

Tool	Package	What it answers	Command used in this case	Cautions
`date`, `timedatectl`	coreutils, systemd	Is the host clock right? Every other timestamp depends on it	`date -u; timedatectl` (Task 4 live response 01)	Compare with a trusted clock: the contained VM was 75 min slow
`uptime`, `who`, `w`, `last`	procps, util-linux	Who is or was logged in, since when	`who -a; w; last -F -n 30`	`last` reads `/var/log/wtmp`; an attacker can edit it, so cross-check with auth.log
`ps`	procps	Running processes and their parents	`ps -eo pid,ppid,user,uid,lstart,etime,stat,args --forest`	A rootkit can hide from ps; confirm with Volatility `linux.pslist`
`/proc/<pid>/*`	kernel	Exact command line, environment, binary, open files of one process	`tr '\0' ' ' < /proc/1534/cmdline`; `tr '\0' '\n' < /proc/1534/environ`; `ls -l /proc/1534/fd`	`environ` showed `SSH_CONNECTION`, which tied the uploader to the SSH session
`lsof`	lsof	Which process has which file or socket open	`lsof -nP -i`; `lsof -nP`	`-n -P`: no DNS or port-name lookups (faster, no network noise)
`ss`	iproute2	Open and listening sockets, with the owning process	`ss -tunap`; `ss -ltnp`	Short-lived connections can be missed; sample in a loop or use tcpdump
`ip`	iproute2	Interfaces, addresses, routes, ARP	`ip -br link; ip -br addr; ip route; ip neigh`	Also used to change config in containment (`ip link set <if> down`)
`iptables`	iptables	Firewall rules and hit counters	`iptables-save`; `iptables -vnL`	Counters prove a rule is matching traffic
`systemctl`	systemd	Services, timers, persistence	`systemctl list-units --type=service --all`; `systemctl list-timers --all`	
`crontab`, `/var/spool/cron`	cron	Scheduled persistence	`crontab -l -u test.nurse`; `ls -la /var/spool/cron/crontabs`	Found the attacker's `*/5` restart job

Tool	Package	What it answers	Command used in this case	Cautions
`find`	findutils	Recently changed, hidden or SUID files	`find /home /tmp /etc /var/spool -xdev -newermt "-3 hours" -type f -printf ...`	Reading files changes atime; collect `stat` first
`ir_live_response.sh`	this project	All of the above, in order of volatility, output to RAM, each output hashed	`ssh ir-analyst@host 'sudo sh -s' < ir_live_response.sh`	Writes only to `/dev/shm`; pull with `sudo tar -C /dev/shm/lr -cf - .`

2. Network capture and analysis

Tool	Package	What it answers	Command used in this case	Cautions
`tcpdump`	tcpdump	Packets on one interface, with a BPF filter	`timeout 25 tcpdump -i enp0s1 -nn -s0 -U -w /dev/shm/IR1-2026-002_live.pcap host 10.78.99.66`	Capture to RAM or a remote host, never to the suspect disk
`tshark` / Wireshark	tshark, wireshark	Protocol decode, filters, statistics	`tshark -r nic0.pcap -Y 'http.request.method==POST && ip.dst==10.78.99.66' -T fields -e frame.time_epoch` ; `tshark -q -z conv,tcp`	Wireshark capture filters (Task 1): `tcp port 22`, `host <ip>`
`dumpcap` + saved capture filters	wireshark-common	Which capture filters the analyst uses; are they valid BPF?	`~/.config/wireshark/cfilters` (7 named filters, Task 1); `dumpcap -i enp0s1 -f '<filter>' -d` compiles each	Wireshark GUI and tshark both read this file; a filter that does not compile fails here, not mid-incident
`capinfos`	wireshark-common	Packet count, first and last time of a capture	`capinfos -c -a -e file.pcap`	Use it to state the capture window in the report
NIC trace (hypervisor)	VirtualBox / QEMU	Traffic seen by the virtual NIC, outside the guest's control	VirtualBox `--nictrace1 on`; lab: QEMU `-object filter-dump,netdev=n0,file=nic0.pcap`	Cannot be tampered with from inside the guest

3. Memory

Tool	Package	What it answers	Command used in this case	Cautions
`gcore`	gdb	Memory image of one process	`gcore -o /dev/shm/syncd 1534`	Briefly pauses the process; run before killing it

Tool	Package	What it answers	Command used in this case	Cautions
VM memory dump	VirtualBox / QEMU	Full RAM of the guest, no guest tools needed	VirtualBox `debugvm <vm> dumpvmcore`; lab: QMP `dump-guest-memory`	1 GiB per 1 GiB RAM; keep outside git
Volatility 3	pip `volatility3` (2.7.0)	Processes, sockets, files, kernel integrity from a RAM image	`vol -s /opt/symbols -f mem.elf linux.pslist.PsList` (also `pstree`, `psaux`, `envars --pid`, `lsof --pid`, `sockstat`, `proc.Maps --pid --dump`, `bash`, `lsmod`, `check_modules`, `check_syscall`, `malfind`, `kmsg`)	Needs a symbol table for the **exact** kernel. Fails on 5-level paging (use `-cpu max,la57=off` / normal VirtualBox CPU)
`dwarf2json`	Volatility Foundation (Go)	Builds that symbol table (ISF) from a debug kernel	`dwarf2json linux --elf vmlinux-6.1.0-54-cloud-amd64 > isf.json; xz isf.json`	Debug kernel from `linux-image-<ver>-dbg`

4. Disk acquisition and file-system analysis

Tool	Package	What it answers	Command used in this case	Cautions
Disk seizure (hypervisor)	VirtualBox / qemu-utils	Exact disk content at a snapshot	VirtualBox `clonemedium disk <vdi> case.raw --format RAW`; lab: `qemu-img convert -U -l snapshot.name=ir-contained -O raw`	Seize from a snapshot, not the live disk
`dc3dd`	dc3dd	Forensic image with built-in hashing and a log	`dc3dd if=/seized/case.raw of=/bigev/case_vda.dd hash=sha256 log=dc3dd_acquisition.log`	Needs a real output file; it will not write to a pipe or FIFO. Attach the source read-only
`dcfldd`, `ewfacquire`	dcfldd, ewf-tools	Alternatives: stream to stdout (dcfldd), compressed E01 with case metadata (ewfacquire)	`ewfacquire -t case -f encase6 /dev/sdX`	E01 is the court-standard container; raw was used here for simplicity
`mmls`	sleuthkit	Partition layout and offsets	`mmls case_vda.dd` (root fs at sector 262144)	
`fsstat`	sleuthkit	File-system type, last mount, journal	`fsstat -o 262144 case_vda.dd`	

Tool	Package	What it answers	Command used in this case	Cautions
`fls`	sleuthkit	File listing, including deleted entries; body file for timelines	`fls -r -p -o 262144 case_vda.dd`; `fls -r -m / -o 262144 case_vda.dd > body`	Deleted entries are marked `*`
`ifind`, `icat`, `istat`	sleuthkit	Path to inode, file content, inode metadata (MAC + birth times)	`ifind -n /var/log/auth.log -o 262144 img`; `icat -o 262144 img 32027 | sha256sum`; `istat -o 262144 img 32027`	Read-only on the image
`mactime`	sleuthkit	Body file to a sorted MAC timeline	`mactime -b body -z UTC -d -y 2026-11-02`	`-y` = ISO dates; blank date means "same as row above"
`sha256sum`	coreutils	Integrity of every item, at every hand-off	`sha256sum file`; `shasum -a 256 -c index.txt` (macOS)	Hash at the source and at the destination

5. Logs and auditing

Tool	Package	What it answers	Command used in this case	Cautions
`auditd` / `auditctl`	auditd	Kernel-level record of file reads, identity changes, every command a user runs	`auditctl -w /srv/ehr/clinical -p r -k ehr-read`; `-a always,exit -F arch=b64 -S execve -F auid>=1000 -k user-exec`	Serial numbers restart each boot: join records on time + serial
`ausearch`	auditd	Query the audit log	`ausearch -k ehr-read -i`; `ausearch -ua 1003 -i`	`-i` turns uids and syscalls into names
rsyslog / `journalctl`	rsyslog, systemd	auth.log, kern.log, application logs	`journalctl -u ssh`; `grep IR-BLOCK /var/log/kern.log`	sshd `LogLevel VERBOSE` logs key fingerprints for cross-host correlation
Wazuh manager + agent	wazuh-manager 4.9.2, wazuh-agent 4.9.2	Central collection, detection (custom rules 100210-100253), FIM, alerting (Tasks 1-6)	`agent_control -l` / `-i 001`; `wazuh-logtest -v`; `grep '"id":"100221"' alerts.json`	The agent on parrot-vm kept reporting during quarantine through one forwarded port (Task 3 C6e). A macOS agent needs admin rights, so Mac logs were shipped as files (`ship_file`, which restarts logcollector if it dies under emulation)

Tool	Package	What it answers	Command used in this case	Cautions
`build_timeline.py`	this project	Super-timeline from macOS and Parrot sources, in UTC	`python3 analysis/build_timeline.py`	Normalise every source to UTC first

6. Containment and recovery

Tool	Package	What it does	Command used in this case	Cautions
`iptables`	iptables	Block, log and isolate	`iptables -I OUTPUT 1 -d 10.78.99.66 -j IR-EGRESS-BLOCK`; `iptables -P INPUT DROP`	Apply over an out-of-band channel (console or guestcontrol) so a mistake cannot lock you out
`usermod`, `gpasswd`, `passwd`, `chage`	passwd	Lock accounts, remove groups, check status	`usermod -L -e 1 test.nurse`; `gpasswd -d test.nurse clinical`; `passwd -S`; `chage -l`	`passwd -S` shows `L` for an account with no password, but **SSH keys still work**. Expire the account (`-e 1`) and remove its keys
`pkill`, `systemctl`	procps, systemd	Kill processes; stop, mask or disable services	`pkill -KILL -u test.nurse`; `systemctl stop cron && systemctl mask cron`	Kill only after memory is preserved
`dpkg -V`	dpkg	Were packaged binaries modified?	`dpkg -V | awk '$2 != "c"'`	Built into Debian and Parrot; no extra install
VBoxManage	VirtualBox	Snapshots, link state, NIC network, guest control	`snapshot take/restore`, `controlvm setlinkstate1 off`, `guestcontrol run` (lab: `vmctl.py`)	Snapshot before every irreversible step
`irtrack.py`	this project	Incident records, status, actions, timeline, evidence	`irtrack.py list`; `irtrack.py report IR1-2026-002`	Status only moves forward; every change is time-stamped

7. Which procedure uses which tools

Procedure (playbook)	Tools
Triage (section 2)	date, ausearch, ss, ps, tcpdump
RB-02 C2 preserve volatile data	tcpdump, ps, ss, lsof, /proc, crontab
RB-02 C3 / C6 block and isolate	iptables, ip, VBoxManage setlinkstate
RB-02 C4 memory	gcore, VBoxManage debugvm dumpvmcore
RB-02 C5 files / section 4 evidence	stat, cp -a, sha256sum, tar, ssh
Disk acquisition	VBoxManage clonemedium, dc3dd, sha256sum

Procedure (playbook)	Tools
Analysis	Volatility 3, Sleuth Kit, mactime, tshark, build_timeline.py
Recovery	VBoxManage snapshot restore, usermod, gpasswd, iptables-restore, systemctl, dpkg -V, validate_recovery.sh